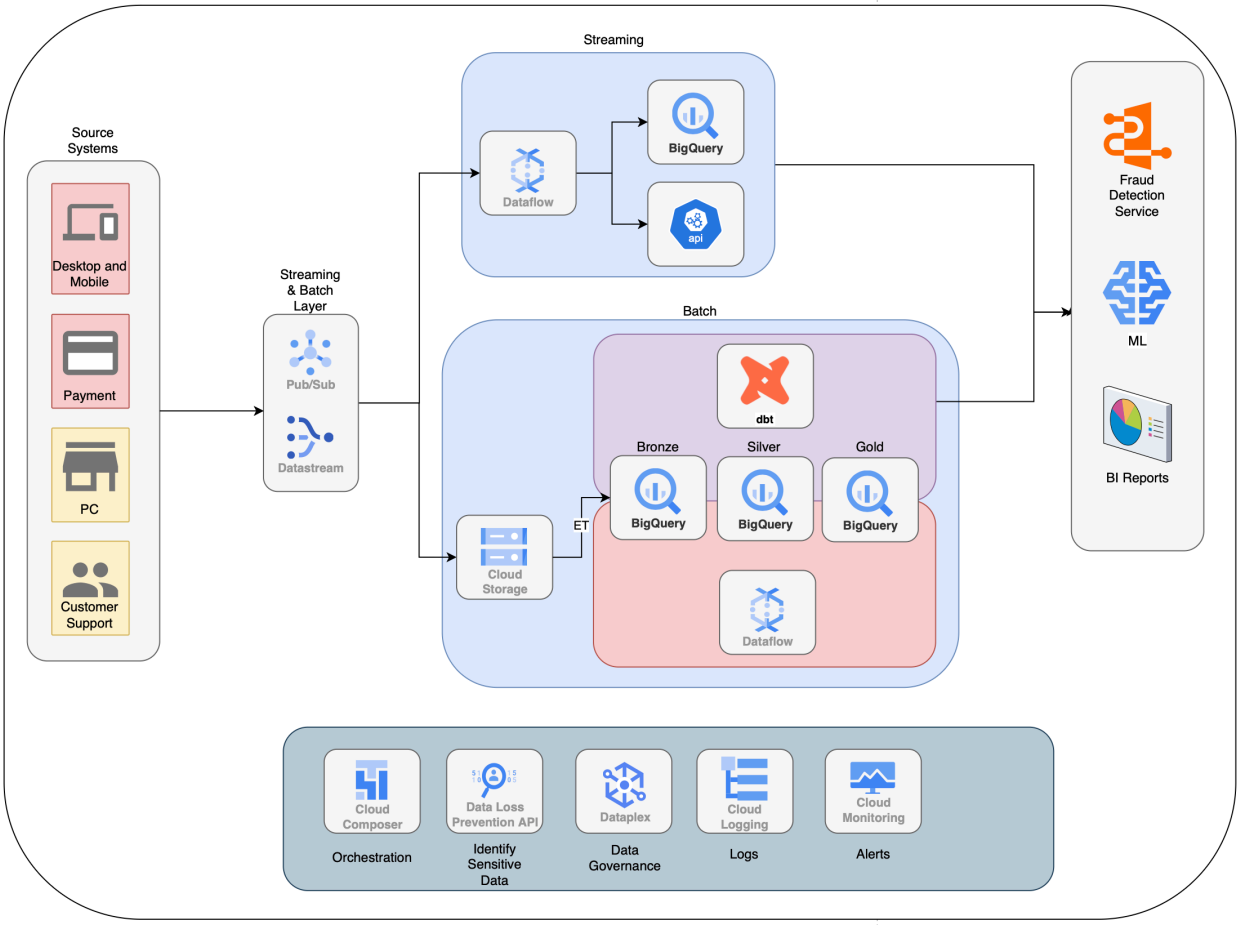
ShopStream Data Pipeline System Design Document

1. Overview

This document outlines the system design for ShopStream's data pipeline, aiming to analyze user behavior, improve conversion rates, and detect potential fraud. The design addresses the need for real-time processing of clickstream events, order transactions, product catalog updates, and customer support interactions, enabling unified customer profiles, real-time fraud detection metrics, daily business intelligence reports, and efficient historical data querying for data scientists.

2. Architecture Diagram

The following diagram illustrates the proposed data pipeline architecture, leveraging Google Cloud Platform (GCP) services for scalability, reliability, and cost-effectiveness.

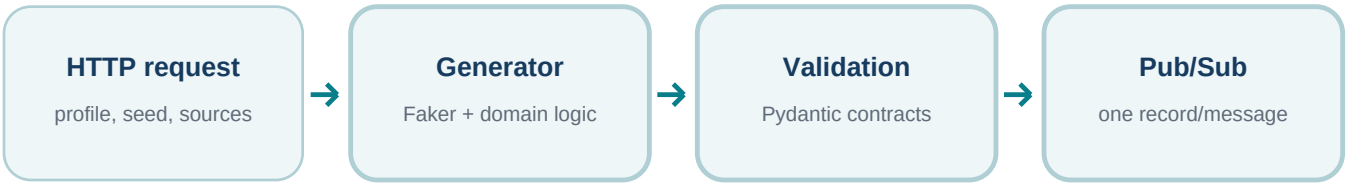


Implemented Synthetic Data Generation

Current portfolio implementation - bounded, reproducible, and emulator-first

What is implemented

The conceptual source systems shown in the architecture diagram are represented in this portfolio by one HTTP Cloud Function. It creates a coherent retail dataset dynamically instead of reading fixed local sample files.



Default and targeted behavior

Request	Generated internally	Published
No sources field	Complete related dataset	All four sources
sources: [clickstream]	Users, products, transactions, clicks	Clickstream only
Multiple selected sources	Minimum supporting entities	Selected sources

Generated sources and contracts

- Product catalog JSON -> product-catalog
- Sales transaction CSV -> sales-transactions
- Clickstream event JSON -> clickstream-topic
- Customer-support ticket JSON -> customer-support-topic
- Pub/Sub attributes carry source, schema_version, event_time, and seed.

Validated locally at the real emulator boundary

A seeded small profile published 135 messages across all four topics and the messages were pulled from local subscriptions. A targeted catalog request published two catalog messages and zero clickstream messages.

Boundary: local function + Pub/Sub emulator. Live GCP deployment is not asserted by this validation.

3. Architecture Explanation

3.1. Data Sources and Ingestion

ShopStream receives data from various sources:

- **Desktop and Mobile Apps:** Generate real-time clickstream events.
- **Payment System:** Provides order transaction data.
- **PC (Product Catalog):** Supplies product catalog updates.
- **Customer Support:** Generates customer support interaction data.

All real-time data from these sources are ingested into **Pub/Sub**, a fully managed asynchronous messaging service, which acts as a central ingestion point (Lambda data architecture). Pub/Sub ensures reliable message delivery and decouples data producers from consumers. For batch data or larger files, **Cloud Storage** is utilized as a landing zone.

(Optional) Datastream can be used for Change Data Capture (CDC) from transactional databases (e.g., payment system, product catalog) to stream changes directly to BigQuery or Google Cloud Storage.

3.2. Processing Layers

The architecture incorporates both streaming and batch processing layers to handle diverse data requirements.

3.2.1. Streaming Layer

- **Dataflow:** A fully managed service for executing Apache Beam pipelines, used for real-time processing of clickstream and other streaming events from Pub/Sub. Dataflow handles:
  - **Sessionization:** Grouping events into user sessions with a 30-minute inactivity timeout.
  - **Real-time Metrics Calculation:** Calculating metrics like events per minute and unique users per minute.
  - **Anomaly Detection:** Identifying potential anomalies (e.g., >100 events from a single user in 1 minute).
- **BigQuery (Streaming):** Processed real-time data from Dataflow is streamed directly into BigQuery for immediate analysis and consumption by the Fraud Detection Service. Another option is that Dataflow sends the data after processing it to the fraud detection API service. It depends on the requirements.

3.2.2. Batch Layer

- **Cloud Storage:** Acts as the data lake, storing raw and semi-processed data from various sources. Data from Pub/Sub can be archived here, and batch data files (e.g., historical transactions, large product catalog updates) are landed here in a partitioned way (dt=yyyy-mm-dd).
- **BigQuery (Bronze, Silver, Gold Layers):** BigQuery is used as the data warehouse, organized into three layers:
  - **Bronze Layer:** Data is accessed via an external table, so the data isn't stored here but on cloud storage. BigQuery only accesses the data and makes it accessible in a tabular way.
  - **Silver Layer:** Contains cleaned, transformed, and conformed data. Data quality checks are applied, and data is structured for easier analysis. This layer might implement Slowly Changing Dimensions (SCD Type 2) for customer profiles.
  - **Gold Layer:** Houses highly aggregated and denormalized data, optimized for specific business intelligence reports and data science queries.
- **(Option 1) dbt:** Used for managing and executing modular SQL transformations within BigQuery. It enables version control, testing, documentation, and dependency management, ensuring data quality and consistency across the Bronze, Silver, and Gold layers.
- **(Option 2) Dataflow (Batch):** Can be used for complex batch transformations that are difficult to achieve with SQL in Dataflow, or for processing large volumes of historical data from Cloud Storage before loading into BigQuery.

The option that best suits it will depend on the team's skill, size, project requirements, data sources, etc.

3.3. Storage Solutions

- **Pub/Sub:** Message queue for real-time event ingestion.
- **Cloud Storage:** Scalable and durable object storage for raw data, data lake, and archival.
- **BigQuery:** Serverless, highly scalable, and cost-effective data warehouse for analytical workloads, supporting both streaming and batch data.

3.4. Data Access and Consumption

- **Fraud Detection Service:** Directly consumes real-time metrics and anomaly alerts from the streaming BigQuery tables or from dbt models.
- **ML (Machine Learning):** Utilizes data from both streaming and batch layers (primarily Gold layer in BigQuery) for training and deploying machine learning

models for fraud detection and predictive analytics.
- **BI Reports:** Business intelligence tools connect to the Gold layer in BigQuery to generate daily and ad-hoc reports, providing insights into user behavior and business performance.
- **Data Scientists:** Can query historical data efficiently from the Bronze, Silver and Gold layers in BigQuery for in-depth analysis and model development.

3.5. Supporting Services

- **Cloud Composer (Orchestration for Batch Purposes):** A fully managed Apache Airflow service used to orchestrate complex data pipelines, including batch ETL jobs, data quality checks, and data loading processes.
- **Data Loss Prevention API (Identify Sensitive Data):** Used to scan and redact sensitive data (e.g., PII) across various data stores to ensure GDPR compliance.
- **Dataplex (Data Governance):** Provides capabilities for data discovery, metadata management, and data quality monitoring across the data lake and data warehouse.
- **Cloud Logging (Logs):** Centralized logging for all GCP services, enabling comprehensive monitoring and troubleshooting.
- **Cloud Monitoring (Alerts):** Provides monitoring dashboards and alerting capabilities for tracking pipeline health, performance, and potential issues.

4. Design Decisions

4.1. Handling Late-Arriving Data

- **Streaming:** For the streaming pipeline, Dataflow's windowing capabilities with allowed lateness will be configured. This allows events arriving slightly after their designated window to still be processed within that window, up to a defined grace period. Events arriving beyond this grace period will be handled as "late data" and can be optionally routed to a dead-letter queue for further investigation or re-processing in a batch job.
- **Batch:** In the batch pipeline, late-arriving data for a given day will be processed in the subsequent day's batch run. Incremental processing and idempotent transformations will ensure that re-processing late data does not lead to data duplication or inconsistencies.

4.2. Schema Evolution Strategy

- **Avro/Parquet with Schema Registry:** Data ingested into Cloud Storage and processed by Dataflow will utilize self-describing formats like Avro or Parquet. A schema registry (e.g., Confluent Schema Registry or a custom solution built on Cloud Storage) will be used to manage schema versions.

- **Backward Compatibility:** New schema versions will be designed to be backward-compatible, meaning older consumers can still process data produced with newer schemas. This typically involves adding new, optional fields.
- **Forward Compatibility:** For critical data, forward compatibility will also be considered, allowing newer consumers to process data produced with older schemas. This might involve setting default values for missing fields.
- **BigQuery:** BigQuery inherently supports schema evolution by allowing new columns to be added at the end of a table. Changes like column type modifications or deletions require more careful planning and potentially creating new tables.

4.3. Data Retention and Archival Policies

- **Raw Data (Cloud Storage):** Raw ingested data in Cloud Storage will be retained for a specified period (e.g., 90 days) for re-processing or auditing purposes. Longer-term archival to colder storage classes (e.g., Coldline or Archive) will be implemented for compliance or historical analysis needs.
- **BigQuery (Bronze, Silver, Gold):**
  - **Bronze:** Data in the Bronze layer will mirror the raw data retention policy.
  - **Silver:** Data in the Silver layer will be retained for a longer period (e.g., 1-2 years) to support historical analysis and data science initiatives.
  - **Gold:** Data in the Gold layer, being highly aggregated, will have the longest retention period (e.g., 5+ years) to support long-term trend analysis and reporting.
- **GDPR Compliance:** For data deletion requests (GDPR), a process will be implemented to identify and delete user-specific data across all relevant storage layers (BigQuery, Cloud Storage). This will involve a combination of data masking, pseudonymization, and physical deletion where required.

4.4. Failure Recovery Mechanisms

- **Idempotent Processing:** All processing jobs (both streaming and batch) will be designed to be idempotent, meaning they can be re-run multiple times with the same input without causing unintended side effects or data corruption.
- **Checkpointing and Snapshots (Dataflow):** Dataflow pipelines will leverage checkpointing to periodically save the state of the pipeline. In case of failure, the pipeline can resume from the last successful checkpoint, minimizing data loss and re-processing.
- **Retry Mechanisms:** Automated retry mechanisms will be implemented for transient errors in data ingestion and processing steps.
- **Dead-Letter Queues:** Failed messages or records that cannot be processed due

to data quality issues or unexpected errors will be routed to dead-letter queues (e.g., a separate Pub/Sub topic or Cloud Storage bucket) for manual inspection and re-processing.
- **Automated Restarts and Alerts:** Monitoring systems (Cloud Monitoring) will be configured to detect pipeline failures and automatically trigger restarts where appropriate, along with sending alerts to the operations team.
- **Data Backups:** Regular backups of critical metadata and configurations will be performed. BigQuery tables have built-in time travel capabilities, allowing recovery to a previous state.

4.5. Scaling Strategy for 10x Growth

- **Serverless and Managed Services:** The heavy reliance on serverless and fully managed GCP services (Pub/Sub, Dataflow, BigQuery, Cloud Storage) provides inherent scalability. These services automatically scale their resources up or down based on demand, eliminating the need for manual provisioning.
- **Pub/Sub:** Can handle millions of messages per second and automatically scales its throughput.
- **Dataflow:** Automatically scales the number of workers based on the workload, ensuring efficient processing of increasing data volumes.
- **BigQuery:** Designed for petabyte-scale data warehousing and automatically scales its compute and storage resources to handle growing data volumes and query complexity.
- **Partitioning and Clustering (BigQuery):** BigQuery tables will be partitioned (e.g., by date) and clustered (e.g., by user ID) to optimize query performance and reduce scan costs as data grows.
- **Cost Optimization:** As data scales, continuous monitoring of costs and optimization strategies (e.g., optimizing BigQuery queries, adjusting data retention policies, leveraging cheaper storage classes) will be crucial.

5. Capacity Planning Calculations (Example)

This section provides a high-level example of capacity planning. Detailed calculations would require more specific data profiles and usage patterns.

5.1. Ingestion (Pub/Sub)

- **Requirement:** Handle 1M events/minute during peak hours.
- **Calculation:** 1,000,000textevents/minute=16,667textevents/second.
- **Pub/Sub Capacity:** Pub/Sub can handle significantly higher throughputs, typically scaling to millions of messages per second. This requirement is well within Pub/Sub's capabilities.

- **Message Size:** Assuming an average message size of 1KB.
- **Throughput:** 16,667textmessages/secondtimes1textKB/message=16.67textMB/second.

5.2. Streaming Processing (Dataflow)

- **Requirement:** Process 1M events/minute.
- **Dataflow Sizing:** Dataflow's auto-scaling will handle this. The number of vCPUs and memory required will depend on the complexity of the Dataflow pipeline (e.g., sessionization logic, number of transformations).
- **Example (rough estimate):** For 1M events/minute, a pipeline might initially require 5-10 n1-standard-4 (4 vCPU, 15GB RAM) workers during peak, scaling up as needed.

5.3. Storage (BigQuery)

- **Clickstream Data:**
  - **Volume:** Assuming 1M events/minute, 1440 minutes/day, and 1KB/event (after compression/optimization).
  - **Daily Ingestion:** 1,000,000textevents/minutetimes1440textminutes/day=1.44textbillionevents/day.
  - **Daily Data Volume:** 1.44textbillionevents/daytimes1textKB/event=1.44textTB/day.
  - **Annual Data Volume:** 1.44textTB/daytimes365textdays/year=525.6textTB/year.
- **Retention:** If clickstream data is retained for 2 years in BigQuery Silver layer: 525.6textTB/yeartimes2textyears=1.05textPB.
- **Other Data (Transactions, Product Catalog, Customer Support):** These datasets are typically smaller in volume compared to clickstream. Assuming an additional 10-20% of the clickstream volume for these.
- **Total BigQuery Storage (Silver/Gold):** Approximately 1.2 - 1.3 PB for 2 years of retention. BigQuery's storage is highly scalable and cost-effective at this scale.

5.4. Query Performance (BigQuery)

- **Sub-second latency for fraud detection:** This will require optimized queries on partitioned and clustered BigQuery tables, potentially leveraging materialized views for frequently accessed aggregates.
- **Daily BI Reports:** Queries on the Gold layer will be optimized for performance.
- **Data Scientists:** Queries on Silver/Gold layers will be supported by BigQuery's columnar storage and distributed query engine.

5.5. Cost Considerations (Rough Estimates)

- **Pub/Sub:** Primarily based on message volume and egress. Relatively low cost for the given scale.
- **Dataflow:** Based on compute (vCPU, memory) and data processed. Can be a significant cost driver, requiring careful pipeline optimization.
- **BigQuery:** Based on storage (/TB/month)andqueryvolume(/TB scanned). Storage costs are low, but query costs can add up if not optimized. Free tier for first 1TB of queries per month.
- **Cloud Storage:** Based on storage volume and operations. Very cost-effective for data lake and archival.
- **Cloud Composer:** Based on environment size and uptime. Can be relatively expensive, so careful consideration for orchestration needs is important.
- **Other Services (DLP, Dataplex, Monitoring):** Costs vary based on usage.

Trade-offs between Cost and Performance:

- **Data Retention:** Longer retention in BigQuery (especially Silver/Gold layers) increases storage costs. A trade-off between historical analysis needs and cost will be made.
- **Dataflow Optimization:** Complex Dataflow pipelines might require more resources, increasing costs. Optimization of Beam pipelines for efficiency is crucial.
- **BigQuery Query Optimization:** Poorly written queries can scan large amounts of data, leading to higher query costs. Strict adherence to best practices (partitioning, clustering, selecting only necessary columns) is essential.
- **Service Choices:** While GCP services offer scalability, using managed services generally comes at a higher cost than self-managing open-source alternatives on VMs. The trade-off is reduced operational overhead and increased reliability.

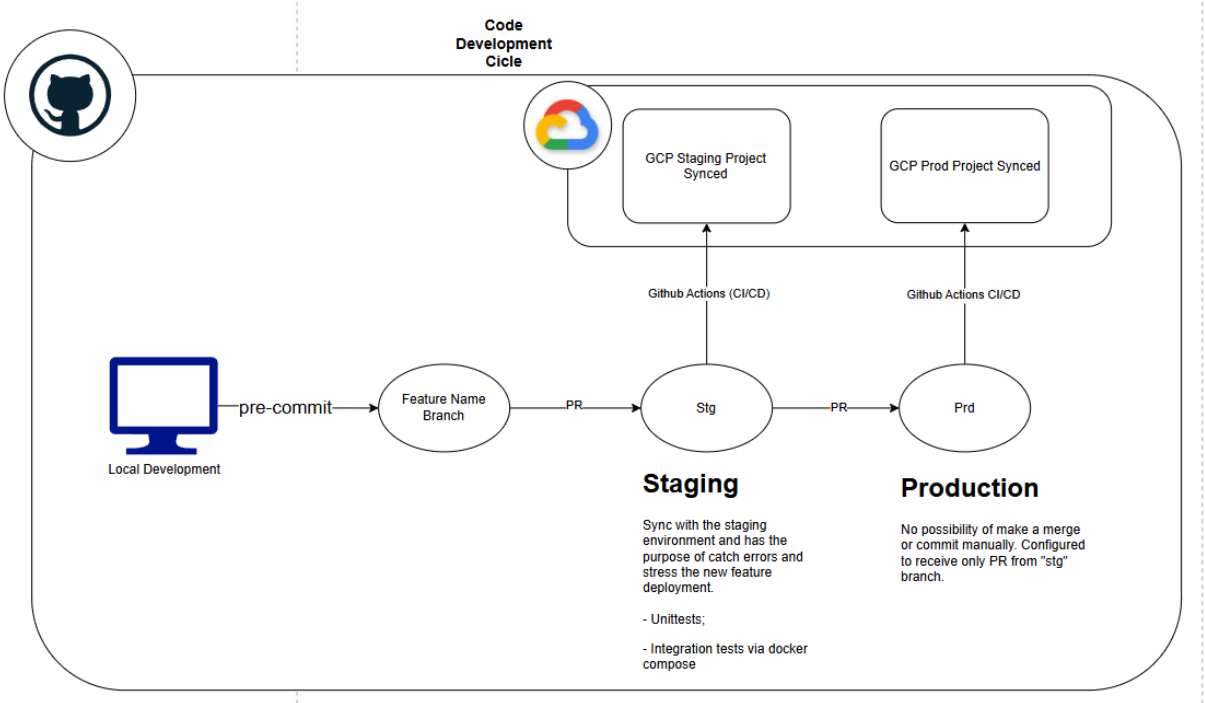
6. Production Readiness

6.1. Deployment Strategy

- **Infrastructure as Code (IaC):** All infrastructure components (Pub/Sub topics, BigQuery datasets/tables, Dataflow jobs, Cloud Storage buckets) will be defined using Terraform. This ensures consistent, repeatable, and version-controlled deployments.
- **CI/CD Pipelines:** Automated CI/CD pipelines (e.g., using Cloud Build or Github Actions) will be implemented for deploying code (Dataflow pipelines, dbt project, Cloud Run Scripts) and infrastructure changes.
    - **Development Environment:** Changes are first deployed to a staging

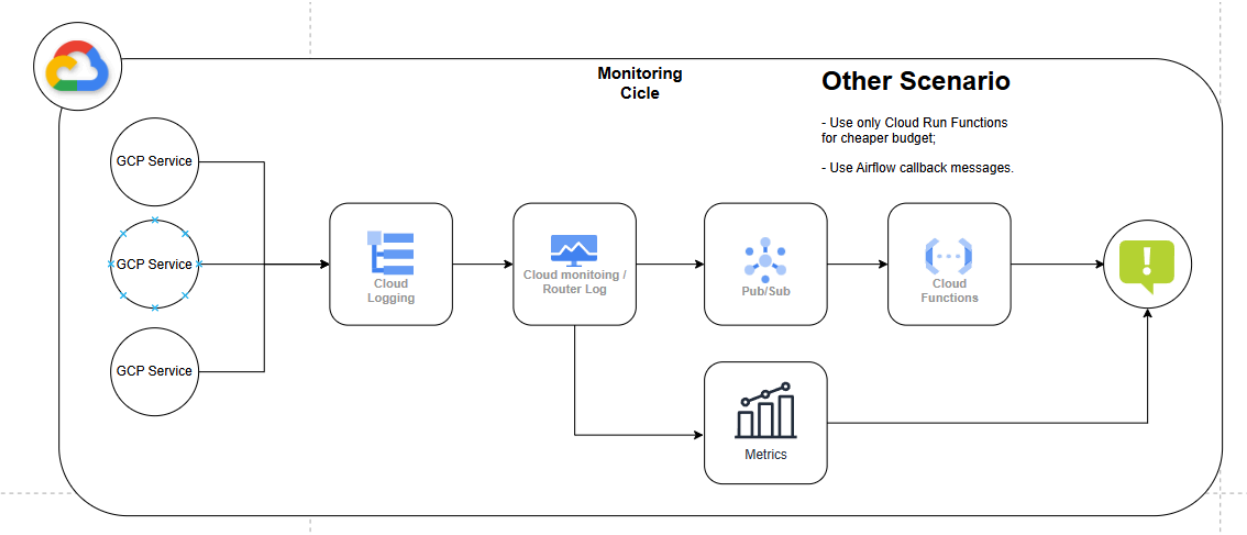
environment for testing.

- **Staging Environment:** After successful development testing, changes are promoted to a staging environment, mirroring production, for integration and performance testing.
- **Production Environment:** Finally, changes are deployed to production after successful staging tests.



- **Blue/Green or Canary Deployments:** For critical updates, blue/green or canary deployment strategies will be considered to minimize downtime and risk.
- **Version Control:** All code and IaC will be stored in a Git repository (e.g., GitHub, Cloud Source Repositories) with meaningful commit messages.

6.2. Monitoring/Alerting Plan



- **Centralized Logging (Cloud Logging):** All application and infrastructure logs will be collected in Cloud Logging. This allows for centralized search, analysis, and export of logs.
- **Metrics Collection (Cloud Monitoring):** Key metrics from all GCP services (Pub/Sub throughput, Dataflow job status, BigQuery query performance, Cloud Storage usage) will be collected and visualized in Cloud Monitoring dashboards. Custom metrics can be defined for application-specific KPIs.
- **Alerting (Cloud Monitoring):** Automated alerts will be configured in Cloud Monitoring for:
  - **Pipeline Failures:** Dataflow job failures, Pub/Sub message backlog exceeding thresholds.
  - **Data Quality Issues:** Alerts triggered by data quality checks (e.g., null values in critical fields, schema mismatches).
  - **Performance Degradation:** Increased latency, decreased throughput, high error rates.
  - **Cost Anomalies:** Unexpected spikes in cloud spending.
- **Notification Channels:** Alerts will be sent to appropriate notification channels (e.g., PagerDuty, Slack, email) based on severity.
- **Dashboards:** Comprehensive dashboards will be created in Cloud Monitoring to provide a real-time overview of the pipeline's health and performance.

6.3. Disaster Recovery Approach

- **Regional Redundancy:** GCP services are inherently highly available and offer regional redundancy. Data in BigQuery and Cloud Storage is replicated across multiple availability zones within a region.
- **Cross-Region Backups/Replication:** For critical data, cross-region replication

or backups will be considered for disaster recovery in case of a regional outage. For example, critical BigQuery datasets could be replicated to a different region.

- **Point-in-Time Recovery (BigQuery):** BigQuery's time travel feature allows querying data from any point within the last 7 days, providing a strong recovery mechanism for accidental deletions or updates.
- **Automated Pipeline Redeployment:** The IaC and CI/CD pipelines will enable rapid redeployment of the entire data pipeline in a different region if a disaster affects the primary region.
- **Data Archival:** Long-term archival of raw data in Cloud Storage (potentially in a separate region) provides a last resort for data recovery.
- **Regular Drills:** Periodic disaster recovery drills will be conducted to test the effectiveness of the recovery procedures and identify any gaps.

6.4. Performance Optimization Ideas

- **BigQuery Query Optimization:**
  - **Partitioning and Clustering:** Already mentioned, crucial for reducing data scanned and improving query performance.
  - **Materialized Views:** For frequently accessed aggregated data, materialized views can pre-compute results, significantly speeding up queries.
  - **Columnar Storage:** Leverage BigQuery's columnar storage by selecting only necessary columns in queries.
  - **Cost Monitoring:** Regularly review BigQuery query costs to identify and optimize expensive queries.
- **Dataflow Pipeline Optimization:**
  - **Minimize Shuffles:** Design Beam pipelines to minimize data shuffles, which are expensive operations.
  - **Efficient Windowing:** Choose appropriate windowing strategies and trigger mechanisms to balance latency and resource usage.
  - **Resource Allocation:** Fine-tune Dataflow worker types and autoscaling parameters based on workload characteristics.
  - **Combine Small Files:** When writing to Cloud Storage, combine small files into larger ones to improve read performance for subsequent batch jobs.
- **Data Format Optimization:** Use efficient data formats like Parquet or Avro, which offer better compression and query performance compared to JSON or CSV, especially for large datasets.
- **Indexing (if applicable):** While BigQuery is a columnar store and doesn't use traditional indexes, for other storage systems (e.g., if a transactional database was part of the query layer), proper indexing would be crucial.
- **Caching:** For frequently accessed BI reports or ML model features, consider

caching mechanisms (e.g., Redis, BigQuery BI Engine) to reduce query latency.
- **Data Quality:** High data quality reduces the need for re-processing and improves the accuracy of analytics, indirectly contributing to performance.